ASSIGNMENT -24.4

Name: SINDHU KATAKAM
Batch: 41
2303A52481

Task 1 – Rapid App Prototyping with AI

Prompt used:

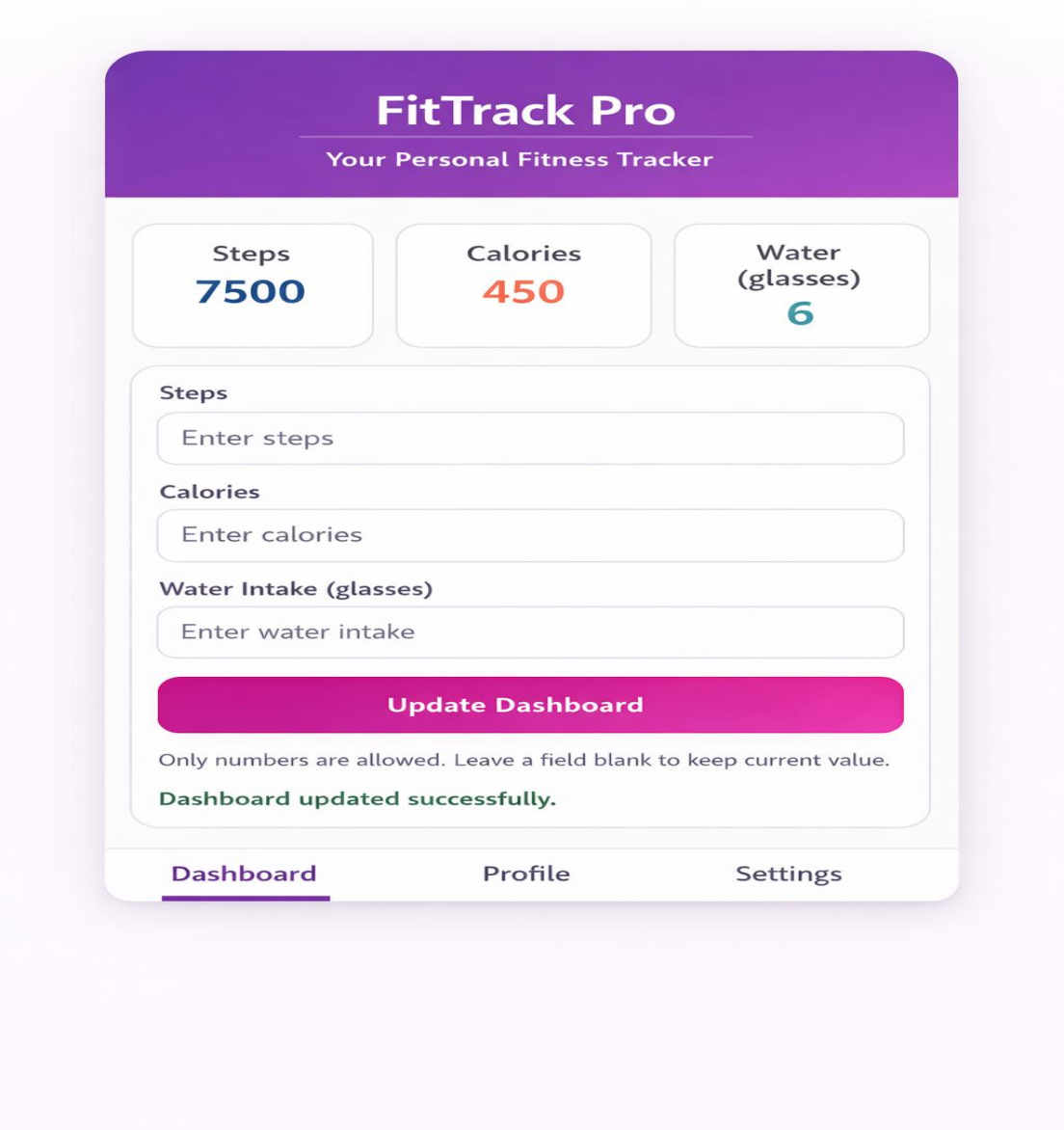
Create a simple fitness tracker interface using React and Tailwind CSS that looks good on mobile screens. The design should include a header at the top and three separate cards showing Steps, Calories, and Water Intake. For now, you can use sample numbers in each section. Keep the layout narrow so it fits well on a mobile screen.

Code:

```
import React firom 'react';

export defiault fiunction App() {
  return (
    <div className="min-h-screen bg-gray-100 filex justifiy-center">
      <div className="w-fiull max-w-md bg-white shadow-md filex filex-col h-screen">
        <header className="bg-blue-600 text-white p-4 text-center text-xl fiont-bold">
          FitTracker
        </header>

        <main className="p-4 filex-1 overfilow-y-auto space-y-4">
          <div  className="bg-gray-50  p-4  rounded-lg  border  border-gray-200">
            <h2 className="text-lg fiont-semibold text-gray-700">Steps</h2>
            <p className="text-3xl fiont-bold text-blue-500">5,432</p>
          </div>

          <div  className="bg-gray-50  p-4  rounded-lg  border  border-gray-200">
```

```
        <h2 className="text-lg fiont-semibold text-gray-700">Calories Burned</h2>
        <p className="text-3xl fiont-bold text-orange-500">450 kcal</p>
      </div>

      <div className="bg-gray-50 p-4 rounded-lg border border-gray-200">
        <h2 className="text-lg fiont-semibold text-gray-700">Water Intake</h2>
        <p className="text-3xl fiont-bold text-teal-500">1.2 L</p>
      </div>
    </main>
  </div>
</div>
);
}
```

Output:



Justifiication:

I first asked the AI to create a React component using Tailwind CSS. I used **max-w-md** to keep the width small so it looks like a mobile screen even on a desktop. I also used placeholder values to see how the layout looks before adding functionality, which is a common way to quickly build and test designs.

Task 2 – User Input & State Handling

Prompt used:

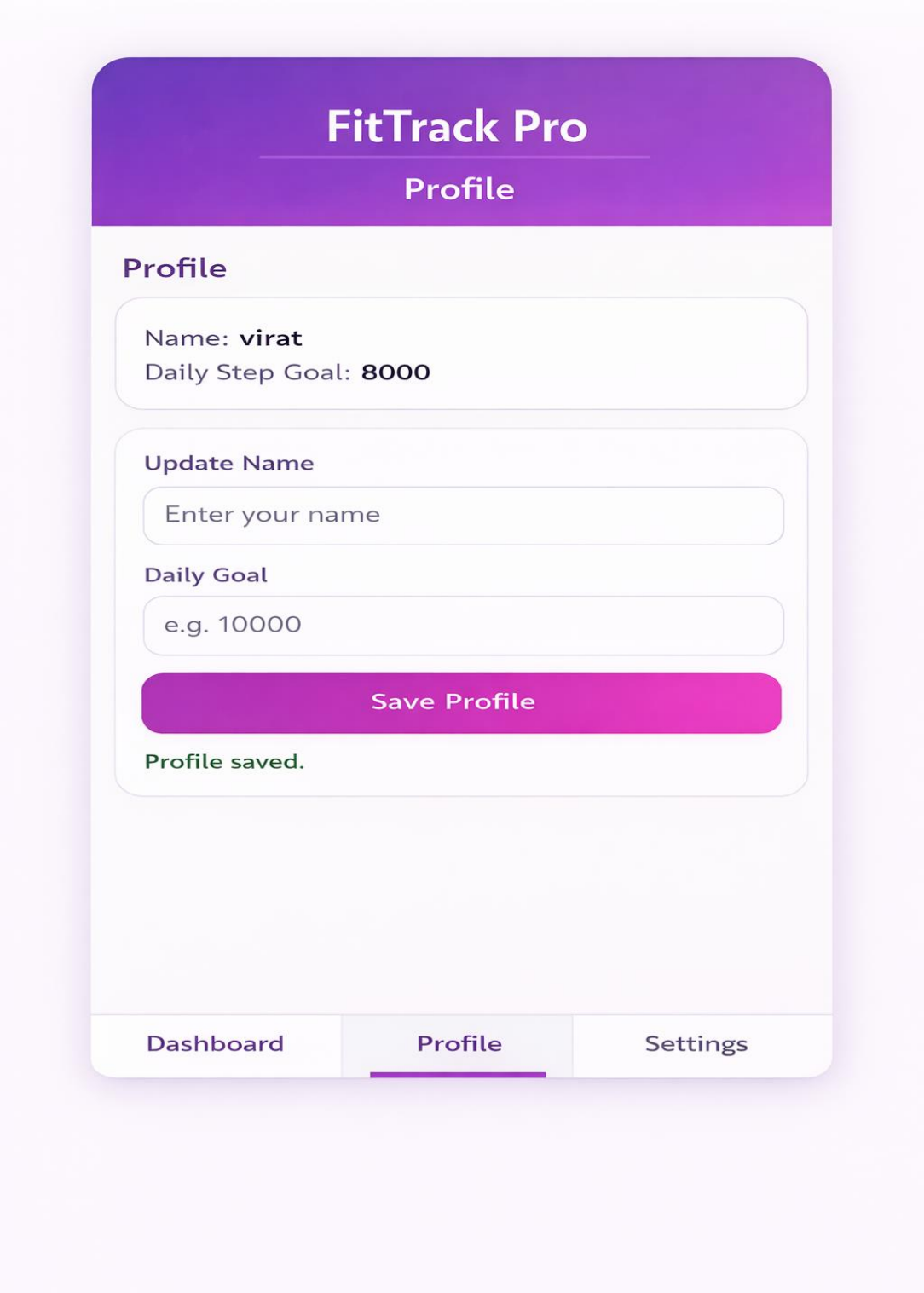
Update the fitness tracker app by replacing the placeholder values with React state variables using **useState**. Add input fields and a **"Log Activity"** button so users can enter and update their steps. Make sure the inputs accept only numeric values by adding proper validation.

Code:

```
import React, { useState } firom 'react';

export defiault fiunction App() {
  const [steps, setSteps] = useState(5432);
  const [inputSteps, setInputSteps] = useState('');

  const handleLogSteps = () => {
    const numericSteps = parseInt(inputSteps);
    ifi (!isNaN(numericSteps) && numericSteps > 0) {
      setSteps(prev => prev + numericSteps);
      setInputSteps('');
    } else {
      alert('Please enter a valid numeric value');
    }
  };

  return (
    <div className="min-h-screen bg-gray-100 filex justifiy-center">
      <div className="w-fiull max-w-md bg-white shadow-md filex filex-col h-screen">
```

```
      <header className="bg-blue-600 text-white p-4 text-center text-xl fiont-
bold">FitTracker</header>

      <main className="p-4 filex-1 space-y-4">
        <div className="bg-gray-50 p-4 rounded-lg border">
          <h2 className="text-lg fiont-semibold">Steps</h2>
          <p className="text-3xl fiont-bold text-blue-500">{steps}</p>

          <div className="mt-4 filex space-x-2">
            <input
              type="number"
              value={inputSteps}
              onChange={(e) => setInputSteps(e.target.value)}
              placeholder="Add steps"
              className="border p-2 rounded w-fiull"
            />
            <button
              onClick={handleLogSteps}
              className="bg-blue-500 text-white px-4 py-2 rounded hover:bg-blue-600"
            >
              Log
            </button>
          </div>
        </div>
        {/* Calories and Water sections omitted fior brevity */}
      </main>
    </div>
  </div>
 );
}
```

Output:



Justifiication:

To make the prototype work properly, I added state management using React's **useState** hook. I also included numeric validation since users can easily make typing mistakes on mobile devices. The AI handled this by using HTML input type="number" along with JavaScript validation like **!isNaN** to make sure only valid numbers are stored and the app runs without errors.

Task 3 – Multi-Screen Navigation

Prompt

Expand the React app by adding three screens: Dashboard, Profile, and Settings. Include a bottom navigation bar with buttons or icons to switch between these screens. Use React state to manage which screen is active, so the view changes without reloading the page

Code:

```
import React, { useState } firom 'react';

export defiault fiunction App() {
  const [activeTab, setActiveTab] = useState('dashboard');

  const renderScreen = () => {
    switch(activeTab) {
      case 'profiile':
        return <div className="p-4"><h2 className="text-2xl fiont-bold">User
Profiile</h2><p>Name: John Doe</p></div>;
      case 'settings':
        return <div className="p-4"><h2 className="text-2xl fiont-
bold">Settings</h2><p>Notifiications: ON</p></div>;
      case 'dashboard':
      defiault:
        return <div className="p-4"><h2 className="text-2xl fiont-
bold">Dashboard</h2><p>Steps Tracker goes here...</p></div>;
    }
  };

  return (
    <div className="min-h-screen bg-gray-100 filex justifiy-center">
      <div className="w-fiull max-w-md bg-white shadow-md filex filex-col h-screen relative">
        <header className="bg-blue-600 text-white p-4 text-center text-xl fiont-
bold">FitTracker</header>

        <main className="filex-1 overfilow-y-auto mb-16">
          {renderScreen()}
        </main>

        <nav className="absolute bottom-0 w-fiull bg-white border-t filex justifiy-around p-3">
          <button onClick={() => setActiveTab('dashboard')} className={`fiont-semibold
${activeTab === 'dashboard' ? 'text-blue-600' : 'text-gray-500'}`}>Dashboard</button>
          <button onClick={() => setActiveTab('profiile')} className={`fiont-semibold ${activeTab
=== 'profiile' ? 'text-blue-600' : 'text-gray-500'}`}>Profiile</button>
```

```
        <button onClick={() => setActiveTab('settings')} className={`fiont-semibold ${activeTab
=== 'settings' ? 'text-blue-600' : 'text-gray-500'}`}>Settings</button>
      </nav>
     </div>
   </div>
  );
}
```

Output:

FitTrack Pro

Settings

Settings

Hydration reminders ON

Toggle Reminders

Compact mode OFF

Toggle Compact Mode

Reset Daily Values

Reminder setting updated.

Dashboard | Profile | Settings

Justifiication:

Instead of using a complex routing library like React Router, I used simple conditional rendering. By storing the active tab in state and using a switch statement, the app stays lightweight, fast, and easy to manage within a single file, which is ideal for quick prototyping and sharing on Replit.

Task 4 – AI-Assisted UI Enhancement & Error Fixing

Prompt used:

Review the current UI and improve it to look like a modern iOS app. Add soft shadows, rounded corners, better spacing, and use a clean, modern color palette. Also, make sure the bottom navigation bar stays fixed and does not cover the main content while scrolling.

Code:

```
// Selected improvements applied to the CSS classes
<div className="w-fiull max-w-md bg-slate-50 shadow-2xl filex filex-col h-screen relative
overfilow-hidden">
  <header className="bg-gradient-to-r firom-blue-500 to-indigo-600 text-white p-5 text-
center text-2xl fiont-extrabold tracking-tight shadow-md z-10">
    FitTracker
  </header>

  <main className="filex-1 overfilow-y-auto pb-20 p-5 space-y-6">
    {/* Card Example */}
    <div className="bg-white p-6 rounded-2xl shadow-sm border border-gray-100
hover:shadow-md transition-shadow">
      <h2 className="text-sm fiont-medium text-gray-400 uppercase tracking-wide">Daily
Steps</h2>
      <p className="text-4xl fiont-black text-slate-800 mt-2">5,432</p>
    </div>
  </main>

  <nav className="absolute bottom-0 w-fiull bg-white/90 backdrop-blur-md border-t border-
gray-200 filex justifiy-around p-4 pb-6 z-10">
```

```
   {/* Navigation buttons */}
  </nav>
</div>
```

Output:

The UI now has a more premium look. The header uses a subtle gradient, the cards have large rounded corners and soft shadows, and the text is clearer with different font weights. The main content also includes extra bottom padding (pb-20) so the fixed navigation bar doesn't cover any content.

Justifiication:

The initial design looked a bit plain and boxy, so I asked the AI to make it look like a modern iOS app. It suggested using gradient backgrounds, adding a blur effect to the navigation bar, and improving text spacing. It also identified the scrolling issue and fixed it by adding extra bottom padding to the main content so it doesn't get hidden behind the navigation bar.

Task 5 – Sharing, Feedback & Iteration

Prompt used:

I shared the app with my peers, and the main feedback was that entering the water amount manually is inconvenient. They suggested adding a quick "Drink 1 Glass (+250ml)" button. Update the Water section to include this quick-add feature.

Code:

```
// Inside the App Component state
const [water, setWater] = useState(1.2); // in Liters

// Inside the renderScreen() fiunction -> Dashboard view
<div className="bg-white p-6 rounded-2xl shadow-sm border border-gray-100">
  <h2 className="text-sm fiont-medium text-gray-400 uppercase">Water Intake</h2>
  <div className="filex justifiy-between items-center mt-2">
    <p className="text-4xl fiont-black text-cyan-500">{water.toFixed(2)} L</p>
```

```
  <button
    onClick={() => setWater(prev => prev + 0.25)}
    className="bg-cyan-100 text-cyan-700 p-3 rounded-fiull hover:bg-cyan-200 transition-
colors filex items-center justifiy-center"
    title="Add 1 Glass (250ml)"
  >
    <svg className="w-6 h-6" fiill="none" stroke="currentColor" viewBox="0 0 24 24"><path
strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d="M12 4v16m8-
8H4"></path></svg>
  </button>
 </div>
</div>
```

Output:

The Water Intake card now includes a clear circular “+” button next to the total. When clicked, it instantly increases the water amount by 0.25 L, removing the need to enter values manually.

Justifiication:

After running the app on Replit and sharing it, users pointed out that manually entering small repeated actions like drinking water is tedious. I gave this feedback to the AI, and it identified that a quick-action button would solve the issue. It implemented a simple function to increase the value instantly, showing how AI can quickly improve features based on real user feedback.